

TO-DO FLOW

1) ./src/components/ToDoApp.js

```
import React, { useState } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { addTodo, toggleTodo, deleteTodo, undo } from '../store/actions';

const ToDoApp = () => {
  const [text, setText] = useState('');
  const todos = useSelector(state => state.get('todos'));
  const dispatch = useDispatch();

  const handleAdd = () => {
    if (text.trim() !== '') {
      dispatch(addTodo(text));
      setText('');
    }
  };

  return (
    <div>
      <h2>Todo List</h2>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
        placeholder="Enter todo"
      />
      <button onClick={handleAdd}>Add Todo</button>
      <button onClick={() => dispatch(undo())}>Undo</button>
      <ul>
        {todos.map(todo => (
          <li key={todo.get('id')}>
            <span
              style={{
                textDecoration: todo.get('completed') ? 'line-through' :
'none',
                cursor: 'pointer'
              }}
              onClick={() => dispatch(toggleTodo(todo.get('id')))}
            >
              {todo.get('text')}
            </span>
            <button onClick={() => dispatch(deleteTodo(todo.get('id')))}>
Delete    </button>
          </li>
        ))}
      </ul>
    </div>
  );
};

export default ToDoApp;
```

2) ./src/store/actions.js

```
// Action types
export const ADD_TODO = 'ADD_TODO';
export const ADD_TODO_SUCCESS = 'ADD_TODO_SUCCESS';
export const TOGGLE_TODO = 'TOGGLE_TODO';
export const DELETE_TODO = 'DELETE_TODO';
export const UNDO = 'UNDO';

// Action creators
export const addTodo = text => ({ type: ADD_TODO, payload: text });
export const addTodoSuccess = text => ({ type: ADD_TODO_SUCCESS, payload: text });
export const toggleTodo = id => ({ type: TOGGLE_TODO, payload: id });
export const deleteTodo = id => ({ type: DELETE_TODO, payload: id });
export const undo = () => ({ type: UNDO });
```

3) ./src/store/epics.js

```
import { ofType } from 'redux-observable';
import { delay, map } from 'rxjs/operators';
import { ADD_TODO, ADD_TODO_SUCCESS } from './actions';

// Simulate async todo adding (delayed by 500ms)
export const addTodoAsyncEpic = action$ =>
  action$.pipe(
    ofType(ADD_TODO),
    delay(500),
    map(action => ({
      type: ADD_TODO_SUCCESS,
      payload: action.payload
    })))
  );
```

4) ./src/store/reducers.js

```
import { Map, List } from 'immutable';
import { ADD_TODO_SUCCESS, TOGGLE_TODO, DELETE_TODO, UNDO } from './actions';

let nextId = 1;

const initialState = Map({
  todos: List(),
  history: List()
});

export const todoReducer = (state = initialState, action) => {
  switch (action.type) {
    case ADD_TODO_SUCCESS:
      return state
        .update('history', h => h.push(state.get('todos')))
        .update('todos', todos =>
          todos.push(

```

```

        Map({
          id: nextId++,
          text: action.payload,
          completed: false
        })
      )
    );

    case TOGGLE_TODO:
      return state
        .update('history', h => h.push(state.get('todos')))
        .update('todos', todos =>
          todos.map(todo =>
            todo.get('id') === action.payload
              ? todo.update('completed', c => !c)
              : todo
          )
        );

    case DELETE_TODO:
      return state
        .update('history', h => h.push(state.get('todos')))
        .update('todos', todos =>
          todos.filter(todo => todo.get('id') !== action.payload)
        );

    case UNDO:
      const prevTodos = state.get('history').last();
      return prevTodos !== undefined
        ? state
          .set('todos', prevTodos)
          .update('history', h => h.pop())
          : state;
    default:
      return state;
  }
};

```

5) App.js

```

import React from 'react';
import TodoApp from './components/TodoApp';

function App() {
  return (
    <div>
      <h1>RxJS + Redux Observable : To-Do Flow</h1>
      <TodoApp />
    </div>
  );
}

export default App;

```

6) App.test.js

```
import { render, screen } from '@testing-library/react';
import App from './App';

test('renders learn react link', () => {
  render(<App />);
  const linkElement = screen.getByText(/learn react/i);
  expect(linkElement).toBeInTheDocument();
});
```

7) index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { Provider } from 'react-redux';
import { createStore, applyMiddleware } from 'redux';
import { todoReducer } from './store/reducers';
import { createEpicMiddleware } from 'redux-observable';
import { addTodoAsyncEpic } from './store/epics';
import App from './App';

// Setup Epic middleware
const epicMiddleware = createEpicMiddleware();

// Create store
const store = createStore(todoReducer, applyMiddleware(epicMiddleware));
epicMiddleware.run(addTodoAsyncEpic);

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

RDX Observables

1. npx create-react-app rdx
2. cd rdx
3. npm install react react-dom react-scripts
npm install redux react-redux
npm install redux-observable rxjs
npm install immutable
4. npm start